

Portear un sistema de intranet como App/Microservicio.

by pepe prime perfected

Durante el proceso de actualización de intranet se decidió cambiar la arquitectura del sistema de varios monolitos a una basada en apps/microservicios. Esto significa que cada sistema de intranet debe ser adaptado a esta arquitectura.

El objetivo de esta guía es aclarar qué es lo que el sistema espera para poder cargar un sistema como app.

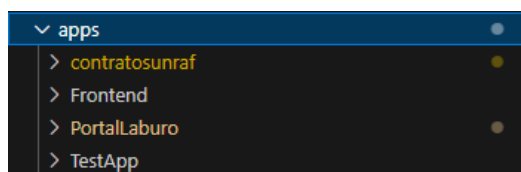
La estructura:

Intranet ahora usa un patrón de diseño conocido como “Kernel compartido”, donde el “front controller” (el controlador principal usado para iniciar el resto del sistema) es modificado para que este solo cargue los recursos necesarios para mostrar una página de un sistema dado en vez de tener todos los archivos cargados al mismo tiempo.

Es decir, el kernel **solo carga la aplicación necesaria para renderizar la página** y una serie de recursos universales. Efectivamente haciendo que todas los sistemas de intranet compartan un kernel y, por ende, una sola instalación de symfony.

Llevado a la práctica, dicha implementación se modifica lo siguiente:

- Se agrega una carpeta llamada “apps”, aquí es donde residirán los sistemas a cargar.



- Se modifica el archivo kernel.php para aceptar el argumento APP_ID, el cual es una variable de entorno definida en el vhost del sistema correspondiente.
- Se agregan las siguientes entradas en el archivo composer.json, que determinan la ubicación y nombre de Clase base para cada aplicación

```
"autoload": {
    "psr-4": {
        "Shared\\": "src/",
        "TestApp\\": "apps/TestApp/src/",
        "Frontend\\": "apps/Frontend/src/",
        "PortalLaburo\\": "apps/PortalLaburo/src/",
        "Contratos\\": "apps/contratosunraf/src/"
    }
}
```

- Se define toda la parte del frontend de intranet como recurso compartido.

```
Shared\:
  resource: '../src/'
  exclude:
    - '../src/DependencyInjection/'
    - '../src/Entity/'
    - '../src/Kernel.php'
    - '../src/Tests/'
    - '../src/Form/'
    - '../src/srcas/'
    - '../src/Defuse/'

# controllers are imported separately to make sure services can be injected
# as action arguments even if you don't extend any base controller class
Shared\Controller\:
  resource: '../src/Controller'
  tags: ['controller.service_arguments']

Shared\Service\:
  resource: '../src/Service'
  tags: ['controller.service_arguments']

Shared\Service\encriptado:
  arguments:
    $params: '%shared.decryption.key%'
```

- Se agrega al .gitignore la carpeta apps para evitar errores de recursión y para poder separar los repositorios de los sistemas con el del intranet principal.

```
###< UnrafApps ###
/apps

### UnrafApps >###
```

¿Por dónde empezar?

Primero hay que copiar los archivos del sistema a portear a la carpeta apps, para eso se clona el repositorio del mismo dentro.

Una vez adquiridos los archivos, hay que crear una nueva rama y hacer que sea la rama a trabajar:

```
git branch "SistemaPortOCualquierNombre"  
git checkout "SistemaPortOCualquierNombre"
```

Se hace esto para evitar colisiones con las ramas main y dev del sistema, ya que el proceso de adaptación es uno que va a llevar tiempo y durante ese plazo puede ser necesario hacer modificaciones al sistema actual, para lo cual es necesario tener estas ramas libres.

Una vez establecido el entorno de trabajo, vamos a empezar con el siguiente paso.

Borrar archivos/carpetas redundantes del sistema a portear

Debido a que el sistema es una aplicación completa de symfony, este tiene carpetas que ya no van a ser necesarias en su versión de microservicio.

A continuación se detalla las carpetas a eliminar y el motivo:

- bin - Contiene binarios de symfony. Se elimina porque el nuevo kernel solo usa un solo binario ubicado en la carpeta bin de la raíz del directorio.
- var - Contiene archivos cache del sistema. Se elimina porque las cache se consolidan en la carpeta var del directorio base.
- vendor - Contiene todas las dependencias de Symfony y los paquetes requeridos para correr el sistema. Se elimina porque todos los paquetes para todos los sistemas se instalan en la carpeta vendor del directorio base.
- .env y .env.local - Dependiendo del sistema, es posible que puedas definir los valores de entorno en el .env del directorio base.
- composer.json y composer.lock - El archivo de dependencias y el archivo de estado de los paquetes instalados, respectivamente. Copiar la información de paquetes pertinentes del primero para poder pegarla en el composer.json base.

- symfony.lock - Similar a composer.lock. Solo se usará el symfony.lock del directorio base.
- rector.php - Herramienta de refactor. Solo se usará desde el directorio base, ejecutar el archivo del directorio de la app no funcionará porque se eliminó la carpeta vendor del mismo.

Una vez limpiado el sistema de archivos de symfony redundantes, se puede proseguir al siguiente paso.

Generar los archivos de carga y reconfiguración de la app

La siguiente tarea es configurar a symfony para que reconozca y cargue los archivos correctos al iniciar la aplicación.

El primer paso es generar un nombre de clase para el sistema, lo cual se realiza modificando el archivo composer.json.

```
"autoload": {
    "psr-4": {
        "Shared\\": "src/",
        "TestApp\\": "apps/TestApp/src/",
        "Frontend\\": "apps/Frontend/src/",
        "PortalLaburo\\": "apps/PortalLaburo/src/",
        "Contratos\\": "apps/contratosunraf/src/"
    }
},
```

Esta sección determina los nombres de clase y las rutas de archivo donde se encuentra el código fuente. Cada línea corresponde a una aplicación o módulo diferente.

Una vez modificado, es necesario ejecutar el siguiente comando para generar los archivos autoload.php necesarios para que se carguen las aplicaciones:

```
composer dump-autoload
```

Esto hará que la aplicación sea cargada cuando la variable de entorno APP_ID sea igual al nombre de la carpeta donde reside.

Desde aquí, los siguientes pasos a seguir son:

- Ir por cada archivo .php del sistema, cambiando el namespace de "App" al nombre de aplicación elegido en el composer.json.

- Para módulos compartidos (como el encriptado) y cambiarles el namespace a “Shared”.
- Eliminar archivos .php que pertenezcan a módulos compartidos (ej. encriptado .php y la carpeta Defuse. Ya que estos se encuentran definidos en el directorio base)
- Cambiar el nombre de clase de “App” al nombre de la app en el archivo Services y security del directorio de configuración de la app
 - Esto asegurará que se cargue el namespace correcto.
- Agregar nuestro directorio de templates al archivo de configuración de twig del sistema.
 - Para usarlos, hay que modificar cada path al template con el prefijo “@NombreDeAppElegido/_path_al_template_”
- Agregar nuestra carpeta assets del modulo al archivo de configuración de AssetMapper.
 - Esto es esencial para que el servidor sirva los archivos js, css y recursos estáticos correctamente.
- Ejecutar el comando “*php bin/console asset-map:compile*” para que el servidor copie y pegue los assets necesarios a la carpeta public.
 - Asegurarse de borrar la carpeta public/assets ANTES de hacer esto.

Manejar archivos estáticos con el asset mapper:

Debido a que estamos declarando cada sistema como microservicio, **cada sistema va a depender** de la carpeta public del frontend.

Esto significa que, si queremos tener acceso a todos los recursos estáticos de uno o varios sistemas, será necesario meterlos todos en la misma carpeta.

Para este fin existe el *asset mapper* el cual automáticamente realiza el proceso anterior por cada sistema.

Utilizarlo en intranet requiere 4 pasos:

1. Generar la entrada correspondiente en el archivo *importmap.php*, ubicado en la base del directorio.

```
//intranet/importmap.php
return [
    'Shared' => [
        'path' => './assets/Shared.js',
        'entrypoint' => true,
    ],
    'contratosMap' => [
        'path' => 'apps/contratosunraf/assets/contratosMap.js',
```

```

        'entrypoint' => true,
    ],
    'NoConformidadesMap.js' => [
        'path' =>
'apps/noconformidades/assets/NoConformidadesMap.js',
        'entrypoint' => true,
    ],
];

```

2. Configurar los directorios accesibles al asset mapper:

```

# intranet/config/packages/asset_mapper.yaml
framework:
    asset_mapper:
        # The paths to make available to the asset mapper.
        paths:
            - assets/
            - public/css
            - public/css.old
            - public/svg
            - public/img
            - public/js
            - public/jsold
            - apps/contratosunraf/assets/
            - apps/noConformidades/assets

```

3. Generar el archivo de importMap ejecutando el script *assetMapFileCreator.php* a través de la consola o creándolo manualmente:

```

PS C:\wamp64\www\intranet> php .\assetMapFileCreator.php
Welcome to the my asset map generator!
Especifique el directorio a leer: apps/noconformidades/assets

Especifique el nombre del mapa (assetMap.js): NoConformidadesMap.js

```

4. Ejecutar el comando *php bin/console asset-map:compile* para que se copien los archivos estáticos a la carpeta public.

Manejar conexiones de base de datos con los archivos *doctrine.yaml*:

En la carpeta config del frontend (config/packages), se definen todas las conexiones a las diferentes bases de datos en el sistema:

```

#intranet/config/packages/doctrine.yaml
doctrine:

```

```

dbal:
  default_connection: shared
  connections:
    shared:
      url: '%env(resolve:DATABASE_URL)%'
      server_version: '8.0.21'
    documentos:
      url: 'mysql://root:unraf2024@127.0.0.1:3306/documentos'
      server_version: '8.0.21'
    contratos:
      url: 'mysql://root:unraf2024@127.0.0.1:3306/contratos'
      server_version: '8.0.21'
    noconformidades:
      url:
'mysql://root:unraf2024@127.0.0.1:3306/no_conformidades'
      server_version: '8.0.21'

```

Como se puede ver, el valor por defecto es el *shared* el cual hace referencia a la base de datos del frontend de intranet.

Para usar otra conexión, es necesario especificarla en la llamada al EntityManager de doctrine:

```
$em = $this->doctrine->getManager("noconformidades");
```

O se puede sobrescribir este valor usando el archivo doctrine.yaml del sistema, ya que estos se cargan después de los recursos compartidos (Bajo el nombre Shared):

```

#intranet/apps/noconformidades/config/packages/doctrine.yaml
doctrine:
  dbal:
    default_connection: noconformidades

```

O también se pueden definir la conexión y sobrescribir el valor en el doctrine.yaml del sistema:

```

#intranet/apps/noconformidades/config/packages/doctrine.yaml
doctrine:
  dbal:
    default_connection: noconformidades
    noconformidades:
      url:
'mysql://root:unraf2024@127.0.0.1:3306/no_conformidades'
      server_version: '8.0.21'

```

Declarar EntityManagers:

A la hora de ejecutar migraciones, el sistema debe tener su propio directorio de migraciones y un entitymanager propio:

Definir migraciones en el doctrine_migrations.yaml del sistema:

```
doctrine_migrations:
  storage:
    table_storage:
      table_name: migration_versions
  migrations_paths:
    'Shared\Migrations': '%kernel.project_dir%/apps/sgp/migrations'
    'SGP\Migrations': '%kernel.project_dir%/apps/sgp/migrations'
  em: SGP
```

Definir EntityManager en la lista de managers (config/packages/doctrine.yaml):

```
dbal:
  default_connection: shared
  connections:
    sgp:
      url: '%env(resolve:DATABASE_URL)%/sgp'
      server_version: '8.0.21'
  orm:
    default_entity_manager: Shared
    SGP:
      naming_strategy:
doctrine.orm.naming_strategy.underscore_number_aware
      connection: sgp
      mappings:
        SGP:
          is_bundle: false
          type: attribute
          dir: "%kernel.project_dir%/apps/sgp/src/Entity"
          prefix: 'SGP\Entity'
          alias: SGP
```

Asegúrate de poner la id del sistema que estás implementando a la hora de ejecutar los comandos *make:migration* y *doctrine:migrations:migrate* usando el parámetro *-i* o *-id*.

Ejemplos:

```
php bin/console make:migration -isgp
```

```
php bin/console doctrine:migrations:migrate -isgp
```

Si todo sale bien, symfony debería preguntarte cual namespace usar para las migraciones al ejecutar make:migration y el nombre de la base de datos correcta al ejecutar doctrine:migrations:migrate

Ejemplos específicos:

A continuación se muestran extractos de código utilizados a modo de ejemplo para el proceso de porteo de un sistema a app/microservicio.

Cada uno de estos extractos tiene un comentario al principio notando donde se debería crear el archivo y que rol cumple.

Agregar los templates del sistema a portear:

```
#apps/contratosunraf/config/packages/twig.yaml
twig:
    paths:
        '%kernel.project_dir%/apps/contratosunraf/templates': Contratos
```

Usar los templates agregados:

```
<!-- apps/contratosunraf/templates/base.html.twig -->
<!DOCTYPE html>
{% include '@Contratos/handler/nav.html.twig' %}
<html>
```

Usar los templates agregados desde php:

```
// apps/contratosunraf/src/Controller/PersonaController.php:289
return $this->render('@Contratos/handler/twigHandler.html.twig',
    $filtro);
```

Agregar un entrypoint al AssetMapper:

```
# importmap.php
return [
    'app' => [
        'path' => './assets/app.js',
        'entrypoint' => true,
    ],
    'contratosMap' => [
        'path' => './assets/contratosMap.js',
        'entrypoint' => true,
    ],
];
```

Importar archivos css con el asset mapper:

```
// assets/contratosMap.js
import "../apps/contratosunraf/public/web/css/menu.css";
```

```
import "../apps/contratosunraf/public/web/css/alerts.css";
import "../apps/contratosunraf/public/web/css/base.css";
import "../apps/contratosunraf/public/web/css/pasosContrato.css";
import "../apps/contratosunraf/public/web/css/contrato.css";
import "../apps/contratosunraf/public/web/css/forms.css";
import "../apps/contratosunraf/public/web/css/nav.css";
import "../apps/contratosunraf/public/web/css/menuUnraf.css";

console.log("Contratos assetMap Loaded.");
```

Cargar el assetmap dentro de un template:

```
{# apps/contratosunraf/templates/base.html.twig #}
{% block javascripts %}
    {% block importmap %}{{ importmap('contratosMap') }}{% endblock %}
{% endblock %}
```

Cambiar el namespace del sistema de App a otro nombre en los archivos de configuración:

```
#apps/contratosunraf/config/ervices.yaml

Contratos\:
    resource: '../src/'
    exclude:
        - '../src/DependencyInjection/'
        - '../src/Entity/'
        - '../src/Kernel.php'
        - '../src/Tests/'
        - '../src/Serviceweb/'
        - '../src/Defuse/'
        - '../src/Controller/'

Contratos\Controller\:
    resource: '../src/Controller/'
    tags: ['controller.service_arguments']

Contratos\Service\:
    resource: '../src/Service'
    tags: ['controller.service_arguments']
```

Una vez hecho esto, cambiar el namespace de “APP” al establecido aqui en todos los archivos php de la carpeta src del microservicio/app.

Ej:

Cambiar de

```
// apps/contratosunraf/src/Controller/ContratoController.php
```

```
namespace App\Controller;
```

A

```
// apps/contratosunraf/src/Controller/ContratoController.php  
namespace Contratos\Controller;
```

En todos los archivos necesarios para correr la aplicación.

Cómo determinar qué aplicación se corre al cargar la página

En dev:

Tan simple como establecer el valor de entorno APP_ID en el archivo .env.local al nombre de la carpeta donde se encuentra instalado el sistema.

En prod:

Crear un host virtual para tu sistema donde establezcas la variable de entorno APP_ID usando el comando SetEnv, con el DocumentRoot puesto a la carpeta public del directorio base:

```
#conf/extra/httpd-vhosts.conf  
<VirtualHost *:80>  
    ServerName contratosmoduletest  
    DocumentRoot "c:/wamp64/www/intranet/public"  
    <Directory "c:/wamp64/www/intranet/apps/contratosunraf/">  
        Options +Indexes +Includes +FollowSymLinks +MultiViews  
        AllowOverride All  
        Require local  
    </Directory>  
    <IfModule fcgid_module>  
        Define FCGIPHPVERSION "8.4.0"  
        FcgidInitialEnv PHPRC ${PHPROOT}${FCGIPHPVERSION}  
        <Files ~ "\.php$">  
            Options +Indexes +Includes +FollowSymLinks +MultiViews +ExecCGI  
            AddHandler fcgid-script .php  
            FcgidWrapper "${PHPROOT}${FCGIPHPVERSION}/php-cgi.exe" .php  
        </Files>  
    </IfModule>  
    SetEnv APP_ID contratosunraf  
</VirtualHost>
```

Notas:

Asegurarse de borrar todas las referencias a “*use Symfony\Component\Routing\Annotation\Route;*”, ya que hacen conflicto con los atributos.

O, si es necesario usar anotaciones, asegúrate de que no hagan conflicto con los atributos del frontend.

Idealmente, deberías especificar que conexión estás usando cuando inyectas un EntityManager, para ello, agrega una entrada en el doctrine.yaml del frontend:

```
#intranet/config/packages/doctrine.yaml
doctrine:
    dbal:
        default_connection: shared
        connections:
            shared:
                url: '%env(resolve:DATABASE_URL)%'
                server_version: '8.0.21'
            documentos:
                url: 'mysql://root:unraf2024@127.0.0.1:3306/documentos'
                server_version: '8.0.21'
            contratos:
                url: 'mysql://root:unraf2024@127.0.0.1:3306/contratos'
                server_version: '8.0.21'
            noconformidades:
                url:
'mysql://root:unraf2024@127.0.0.1:3306/no_conformidades'
                server_version: '8.0.21'
```

Y usa el nombre como parámetro cuando instancias/inyectes un manager:

```
$em = $this->doctrine->getManager("noconformidades");
```

Pero si no hay tiempo para cambiar cada llamada, también se puede definir un override:

```
#intranet/apps/noconformidades/config/packages/doctrine.yaml
doctrine:
    dbal:
        default_connection: noconformidades
```

Esto hará que la app defina su propia conexión por defecto, sobrescribiendo el del archivo base (el cual es ‘shared’). **Solo hacer esto como solución temporal**, si tienes tiempo, **escribe explícitamente cuál conexión quieres hacer al instanciar el manager**.

Los nombres de carpetas/archivos son case sensitive en Debian y case insensitive en windows, así que asegúrese de nombrar archivos de forma consistente usando camelcase y que 2 archivos no tengan el mismo nombre pero con 2 diferentes cases.

Composer por defecto no descargara paquetes que se encuentren afectados por problemas de seguridad (security advisories), por lo cual será necesario actualizar el número de versión del paquete afectado.

```
PS C:\wamp64\www\intranet> composer audit
Found 1 security vulnerability advisory affecting 1 package:
+-----+-----+
| Package           | symfony/http-foundation | | | | | | | | | | | | | | | | |
| Severity          | high                    |
| CVE               | CVE-2025-64500          |
| Title             | CVE-2025-64500: Incorrect parsing of PATH_INFO can lead to limited authorization bypass |
| URL               | https://symfony.com/blog/cve-2025-64500-incorrect-parsing-of-path-info-can-lead-to-limited-authorization-bypass |
| Affected versions | >=2.0.0,<3.0.0|>=3.0.0,<4.0.0|>=4.0.0,<5.0.0|>=5.0.0,<5.1.0|>=5.1.0,<5.2.0|>=5.2.0,<5.3.0|>=5.3.0,<5.4.0|>=5.4.0,<5.4.50|>=6.0.0,<6.1.0|>=6.1.0,<6.2.0|>=6.2.0,<6.3.0|>=6.3.0,<6.4.0|>=6.4.0,<6.4.29|>=7.0.0,<7.1.0|>=7.1.0,<7.2.0|>=7.2.0,<7.3.0|>=7.3.0,<7.3.7 |
| Reported at      | 2025-11-12T11:09:14+00:00 |
+-----+-----+
Found 1 abandoned package:
+-----+-----+
| Abandoned Package | Suggested Replacement |
+-----+-----+
| phpunit/php-token-stream | none |
+-----+-----+
```

Ejemplo:

Los paquetes de Symfony 7.2 fueron afectados por una vulnerabilidad, así que composer no los descargara:

```
"symfony/asset": "7.2.*",
```

Lo ideal es actualizar la definición de versión en el archivo:

```
"symfony/asset": "7.3.*",
```

O deshabilitar el chequeo de raíz (**No hacer esto en prod**):
composer update --no-audit

```
--no-audit--no-audit
```